Song2Vec
a Music Recommender System with Word Embeddings

Elif Ezgi İZGİN

March 30, 2026

1 Problem Definition

Music recommendation is one of the most challenging and important problems for streaming platforms because these platforms aim to financially benefit from the users who spend more time using their services and they need the help of recommending the most relevant tracks that align with the user's listening preferences. In this report, we address *next-song prediction*: given a playlist of N songs, predict the N-th song using the preceding songs as context.

We frame this as an embedding learning problem. Inspired by the Word2Vec framework in natural language processing, we treat playlists as sentences and songs as words. Under this approach, songs that frequently appear together in user created playlists are assumed to share semantic similarity as words appearing in similar contexts share meaning. We train a Skip-gram model with Negative Sampling (SGNS) on the 30Music dataset to learn dense vector representations for each song, referred to as Song2Vec embeddings.

Given these embeddings, next-song prediction is performed using two evaluation approaches.

- **Context Averaging:** The average embedding of all $N-1$ context songs is computed and used as a query to retrieve the most similar songs from the vocabulary.
- **Single Query:** Following [1], only the $(N-1)$-th song is used as the query.

Both approaches produce a ranked list of candidate songs, and performance is measured using Hit Rate at K (HR@K) and Normalized Discounted Cumulative Gain at K (NDCG@K) for $K \in \{5, 10, 20, 50\}$.

2 Dataset

We use the 30Music dataset [2], a collection of user-created playlists retrieved from internet radio stations through the Last.fm API. The dataset contains playlist files in the `.idomaar` format, a tab-separated structure with JSON-encoded fields. Each playlist record stores a sequence of track IDs along with metadata such as title and duration. Track metadata is stored in a separate file, where each track's name is URL-encoded in the format `Artist/_/Title`.

Dataset Statistics

The dataset is split at the playlist level into 37,704 training playlists (80%) and 9,426 test playlists (20%), for a total of 47,130 usable playlists for evaluation. The split is performed before model training, and no track-level information from the test set is used during vocabulary construction or embedding learning. The processed dataset statistics are summarized in Table 1. The playlists with no valid tracks and have less than 2 tracks are discarded from the model.

Table 1: Summary statistics of the processed 30Music dataset.

Statistic	**Value**
Total playlists	47,130
Train playlists (80%)	37,704
Test playlists (20%)	9,426
Total track occurrences	1,601,748
Unique tracks	443,184
Unique artists	60,282
Min playlist length	2
Max playlist length	200
Mean playlist length	33.99
Median playlist length	15.00

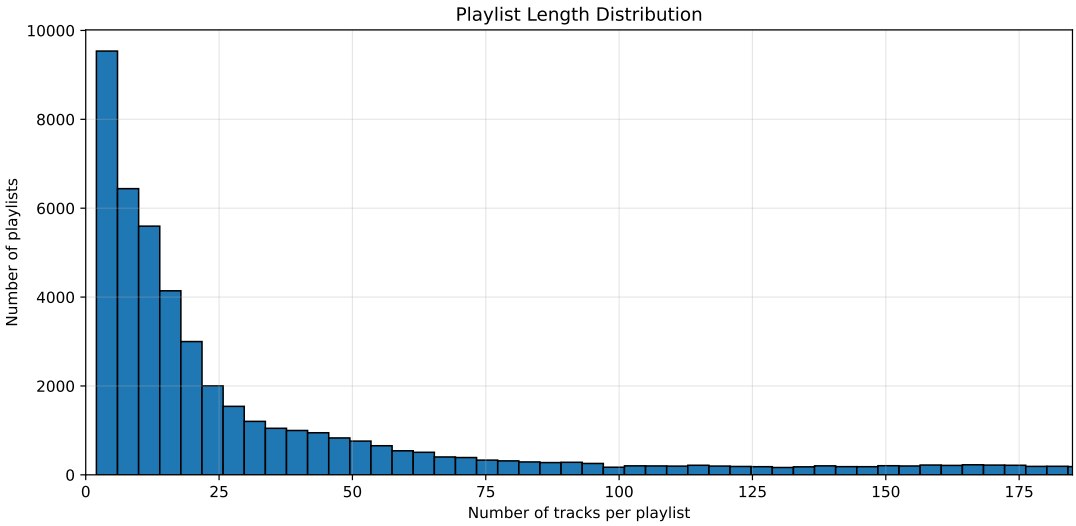

Figure 1: Playlist length distribution. The x-axis is cropped at the 99th percentile for clarity.

The distribution is heavily right-skewed, with most playlists containing fewer than 30 tracks and a long tail extending beyond 150 tracks.

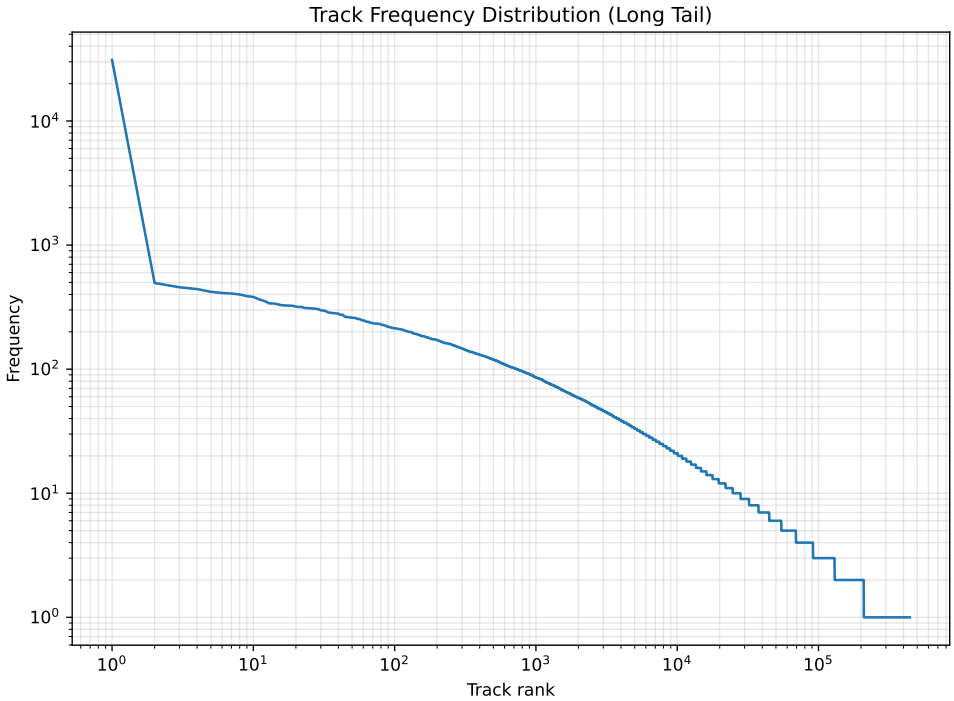

Figure 2: Track frequency distribution on a log-log scale, illustrating the long-tail nature of track popularity.

The plot confirms a power-law distribution: the top-ranked tracks appear tens of thousands of times, while the majority of tracks appear fewer than 10 times. This long-tail property means that a small fraction of tracks dominates playlist co-occurrences, which directly affects embedding quality. Frequent tracks receive rich training signal while rare tracks do not.

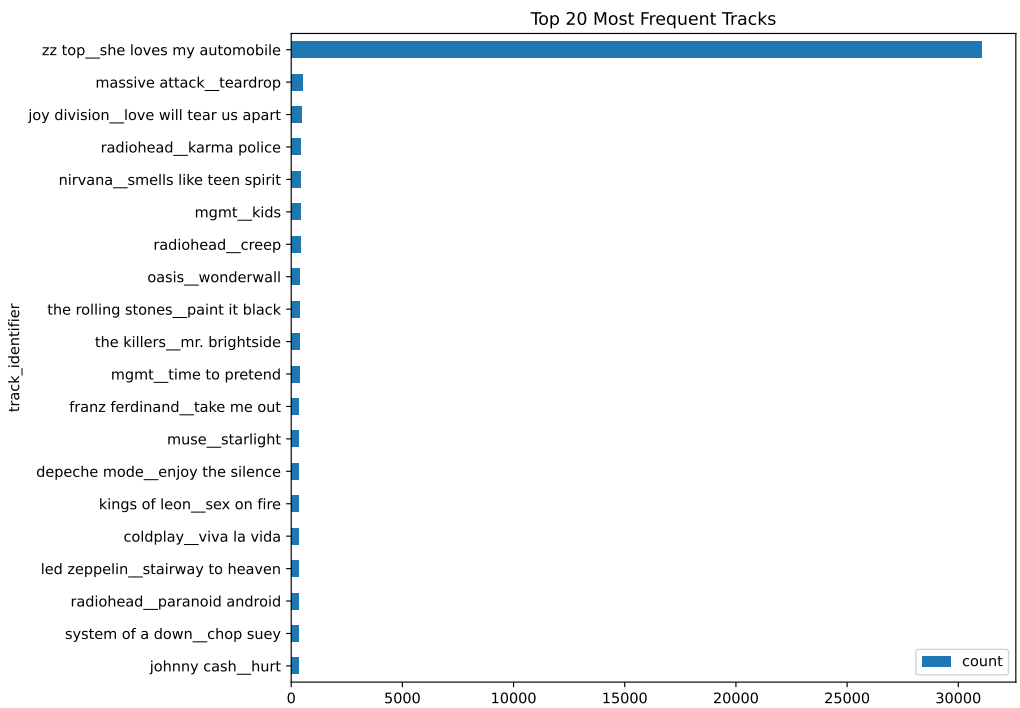

Figure 3: Top 20 most frequent tracks in the dataset.

The most frequent tracks are dominated by alternative and indie rock artists such as Radiohead, MGMT, and Nirvana, with `zz top__she loves my automobile` being a clear outlier at over 30,000 occurrences.

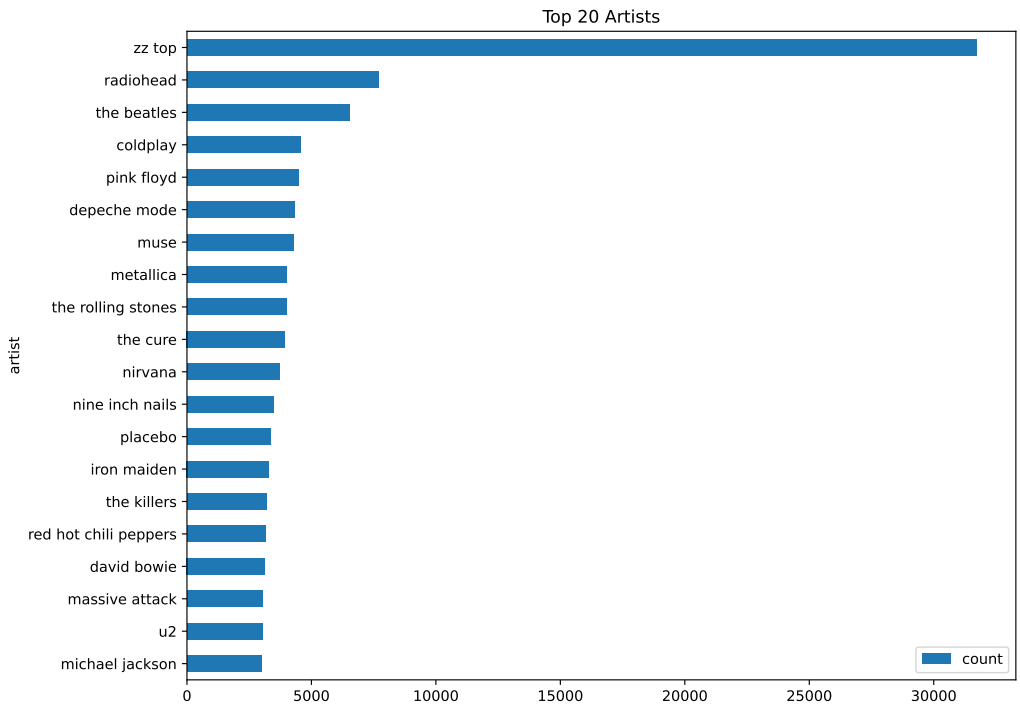

Figure 4: Top 20 most frequent artists by total track occurrences.

Radiohead is the most frequent artist, followed by The Beatles and Coldplay, reflecting the alternative and classic rock preferences of the playlist dataset.

Vocabulary Analysis

The track frequency distribution follows a heavy long tail: 52.88% of unique tracks (234,334 out of 443,184) appear only once across all playlists (hapax legomena). This skew has direct implications for vocabulary pruning. With `min_count=5`, only 15.60% of unique track types are retained, yet these tracks account for 62.75% of all track occurrences (token coverage). Table 2 summarizes coverage statistics across different `min_count` thresholds with all playlists data (train+test).

Table 2: Vocabulary coverage under different `min_count` thresholds.

min_count	Vocab size	Type coverage	Token coverage
1	443,184	100.00%	100.00%
3	129,854	29.30%	75.51%
5	69,140	15.60%	62.75%
10	28,048	6.33%	46.38%

3 Methodology

3.1 Preprocessing

Raw track names are decoded using `urllib.parse.unquote_plus()` and split on the `/_/` separator to extract artist and title fields. Each track is assigned a unique string identifier of the form `artist__title`, where both fields are lowercased, unicode-normalized (NFKC), and stripped of leading/trailing punctuation. Underscores within names are preserved; the double-underscore separator is reserved exclusively for delimiting artist and title.

During playlist parsing, entries with malformed JSON, empty track lists, or non-dictionary object fields (e.g., `[[]]`) are discarded. Playlists with fewer than two valid tracks are also excluded, as next-song prediction requires at least one context song and one target song. Of the raw playlist records, 9,139 are

discarded due to having no resolvable track IDs (including records with `objects=[[]]` or `objects=[]`), and 1,292 are discarded for containing fewer than two tracks, yielding 47,130 usable playlists.

3.2 Reproducibility Evidence

All experiments are fully reproducible. A fixed random seed of 42 is used throughout: for the train/test split, for Word2Vec training, and for t-SNE visualization. All hyperparameter configurations are collected in a single config dictionary in the notebook. Training logs including per-epoch loss and training time are saved to `final_model_training_logs.json`. The trained final model is saved using `model.save()` and can be reloaded without retraining. The executed notebook with all outputs visible is submitted alongside this report.

3.3 Baseline Model

The baseline model is trained using Gensim's Word2Vec implementation with the following configuration: architecture = Skip-gram (sg=1), vector size = 100, window size = 5, min count = 5, negative samples = 5, ns_exponent = 0.75, epochs = 5, workers = 4.

Training completed in 36.1 seconds on the training set of 37,704 playlists, resulting in a vocabulary of 54,651 unique tracks. Figure 5 shows the loss progression across epochs.

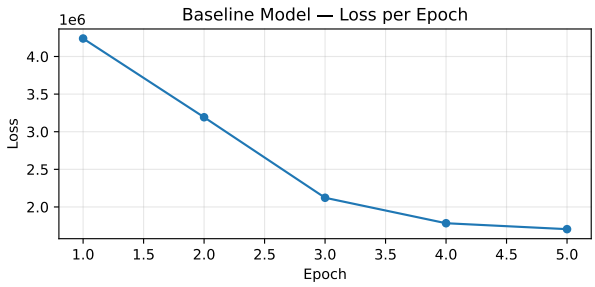

Figure 5: Baseline model loss per epoch.

The loss decreases steadily across epochs, converging by epoch 5. Table 3 reports HR@K and NDCG@K for both evaluation approaches.

Table 3: Baseline model evaluation results.

K	Approach	HR@K	NDCG@K
5	Context Averaging	0.0166	0.0138
10	Context Averaging	0.0249	0.0164
20	Context Averaging	0.0350	0.0190
50	Context Averaging	0.0503	0.0220
5	Single Query	0.0214	0.0143
10	Single Query	0.0331	0.0181
20	Single Query	0.0502	0.0224
50	Single Query	0.0706	0.0265

Single Query outperforms Context Averaging across all K values. This is consistent with the paper approach described in Caselles-Dupré et al. [1]: using the second-to-last song as a single query preserves specificity, whereas averaging all context embeddings produces a diffuse vector that is harder to match against individual tracks.

3.4 Hyperparameter Analysis

3.4.1 Single-Parameter Experiments

We systematically vary each hyperparameter while keeping the others fixed at baseline values. Bold rows indicate the values selected for the subsequent grid search. For each parameter, we report HR@K and NDCG@K for both evaluation approaches.

Table 4: Effect of architecture (sg) on HR@K and NDCG@K. Baseline: sg=1.

sg	Approach	HR@5	HR@10	NDCG@5	NDCG@10
0 (CBOW)	Context Avg.	0.0175	0.0200	0.0151	0.0159
0 (CBOW)	Single Query	0.0057	0.0087	0.0044	0.0053
1 (Skip-gram)	**Context Avg.**	**0.0182**	**0.0258**	**0.0150**	**0.0174**
1 (Skip-gram)	**Single Query**	**0.0227**	**0.0355**	**0.0134**	**0.0175**

Skip-gram (sg=1) consistently outperforms CBOW (sg=0), particularly under the Single Query approach where the gap is largest (HR@10: 0.0355 vs. 0.0087).

Table 5: Effect of window size on HR@K and NDCG@K.

window	Approach	HR@5	HR@10	NDCG@5	NDCG@10
5	Context Avg.	0.0191	0.0245	0.0155	0.0173
5	Single Query	0.0224	0.0375	0.0136	0.0184
10	Context Avg.	0.0209	0.0296	0.0175	0.0203
10	Single Query	0.0268	0.0485	0.0171	0.0241
20	**Context Avg.**	**0.0213**	**0.0321**	**0.0161**	**0.0196**
20	**Single Query**	**0.0328**	**0.0525**	**0.0201**	**0.0264**
40	**Context Avg.**	**0.0238**	**0.0343**	**0.0173**	**0.0207**
40	**Single Query**	**0.0368**	**0.0559**	**0.0247**	**0.0309**

Larger window sizes consistently improve performance, confirming that in playlist-based recommendation, broader co-occurrence context is more informative than in NLP tasks.

Table 6: Effect of number of negative samples on HR@K and NDCG@K.

negative	Approach	HR@5	HR@10	NDCG@5	NDCG@10
5	Context Avg.	0.0191	0.0274	0.0154	0.0180
5	Single Query	0.0231	0.0365	0.0143	0.0187
10	Context Avg.	0.0218	0.0298	0.0175	0.0201
10	Single Query	0.0281	0.0421	0.0174	0.0220
15	**Context Avg.**	**0.0213**	**0.0307**	**0.0175**	**0.0205**
15	**Single Query**	**0.0291**	**0.0448**	**0.0183**	**0.0235**
20	**Context Avg.**	**0.0209**	**0.0294**	**0.0168**	**0.0195**
20	**Single Query**	**0.0308**	**0.0472**	**0.0205**	**0.0258**

Increasing the number of negative samples improves performance up to a point, with diminishing returns beyond 15 under Context Averaging and continued gains under Single Query at 20.

Table 7: Effect of ns_exponent on HR@K and NDCG@K. NLP default is 0.75; recommendation setting tested across a wider range.

ns_exponent	Approach	HR@5	HR@10	NDCG@5	NDCG@10
−1.00	Context Avg.	0.0121	0.0148	0.0112	0.0121
−1.00	Single Query	0.0067	0.0100	0.0038	0.0049
−0.75	Context Avg.	0.0126	0.0162	0.0117	0.0129
−0.75	Single Query	0.0084	0.0154	0.0050	0.0072
−0.50	Context Avg.	0.0139	0.0200	0.0125	0.0144
−0.50	Single Query	0.0134	0.0227	0.0083	0.0113
0.00	Context Avg.	0.0184	0.0260	0.0153	0.0177
0.00	Single Query	0.0221	0.0324	0.0133	0.0166
0.50	Context Avg.	0.0182	0.0238	0.0149	0.0167
0.50	Single Query	0.0224	0.0351	0.0150	0.0191
0.75	**Context Avg.**	**0.0175**	**0.0238**	**0.0145**	**0.0165**
0.75	**Single Query**	**0.0221**	**0.0355**	**0.0138**	**0.0180**
1.00	**Context Avg.**	**0.0189**	**0.0263**	**0.0151**	**0.0174**
1.00	**Single Query**	**0.0204**	**0.0375**	**0.0141**	**0.0196**

Contrary to the recommendation of Caselles-Dupré et al. [1], negative values of ns_exponent perform worse on our data; positive values (0.75 and 1.0) yield the best results.

Table 8: Effect of min_count on HR@K and NDCG@K. Higher min_count reduces vocabulary size and increases skipped test cases.

min_count	Skipped (ctx)	Approach	HR@5	HR@10	NDCG@5	NDCG@10
3	3,964	Context Avg.	0.0123	0.0178	0.0103	0.0121
3	5,413	Single Query	0.0130	0.0229	0.0086	0.0118
5	**4,970**	**Context Avg.**	**0.0193**	**0.0251**	**0.0157**	**0.0176**
5	**6,436**	**Single Query**	**0.0211**	**0.0314**	**0.0144**	**0.0177**
7	5,614	Context Avg.	0.0249	0.0336	0.0204	0.0231
7	7,041	Single Query	0.0277	0.0419	0.0172	0.0218
10	6,248	Context Avg.	0.0330	0.0441	0.0262	0.0297
10	7,581	Single Query	0.0363	0.0656	0.0237	0.0332

Higher min_count yields better HR@10 at the cost of vocabulary coverage; we select min_count=5 as a fixed parameter to balance embedding quality and test set coverage.

Table 9: Effect of vector_size on HR@K and NDCG@K.

vector_size	Approach	HR@5	HR@10	NDCG@5	NDCG@10
50	Context Avg.	0.0193	0.0260	0.0156	0.0178
50	Single Query	0.0217	0.0331	0.0136	0.0173
100	**Context Avg.**	**0.0173**	**0.0242**	**0.0142**	**0.0164**
100	**Single Query**	**0.0184**	**0.0304**	**0.0113**	**0.0153**
200	Context Avg.	0.0189	0.0238	0.0155	0.0171
200	Single Query	0.0154	0.0291	0.0094	0.0139
300	Context Avg.	0.0184	0.0247	0.0150	0.0170
300	Single Query	0.0191	0.0308	0.0117	0.0155

Vector size has a relatively small effect on performance; 50 and 100 perform similarly, while larger dimensions (200, 300) show slightly poorer performance, suggesting that 100 dimensions are sufficient to capture the co-occurrence structure of this dataset.

Table 10: Effect of number of training epochs on HR@K and NDCG@K.

epochs	Approach	HR@5	HR@10	NDCG@5	NDCG@10
5	Context Avg.	0.0182	0.0247	0.0150	0.0171
5	Single Query	0.0247	0.0361	0.0154	0.0190
10	**Context Avg.**	**0.0229**	**0.0330**	**0.0177**	**0.0209**
10	**Single Query**	**0.0371**	**0.0559**	**0.0236**	**0.0295**
20	**Context Avg.**	**0.0227**	**0.0341**	**0.0168**	**0.0205**
20	**Single Query**	**0.0391**	**0.0612**	**0.0262**	**0.0331**
30	Context Avg.	0.0213	0.0332	0.0150	0.0188
30	Single Query	0.0361	0.0582	0.0248	0.0318
50	Context Avg.	0.0195	0.0285	0.0134	0.0163
50	Single Query	0.0401	0.0579	0.0269	0.0327

Performance improves up to 20 epochs and then improvement slows down or slightly decreases, indicating convergence. Epochs beyond 20 do not provide meaningful gains and risk overfitting.

3.4.2 Grid Search

Based on the single-parameter experiments, we select the most promising values for a grid search: `window` $\in \{20, 40\}$, `negative` $\in \{15, 20\}$, `ns_exponent` $\in \{0.75, 1.0\}$, and `epochs` $\in \{10, 20\}$, with `vector_size=100`, `min_count=5`, and `sg=1` fixed. This yields 16 combinations. Table 11 reports all results sorted by average HR@10.

Table 11: Grid search results sorted by average HR@10. The best configuration is highlighted in bold.

window	neg.	ns_exp.	epochs	ctx HR@10	ctx HR@50	sng HR@10	sng HR@50	avg HR@10
40	**20**	**0.75**	**20**	**0.0505**	**0.1228**	**0.0816**	**0.1535**	**0.0660**
40	20	1.00	20	0.0478	0.1185	0.0813	0.1455	0.0645
40	15	0.75	20	0.0467	0.1131	0.0783	0.1441	0.0625
20	20	0.75	20	0.0415	0.0985	0.0793	0.1435	0.0604
40	15	1.00	20	0.0433	0.1120	0.0739	0.1411	0.0586
20	15	1.00	20	0.0395	0.0952	0.0759	0.1341	0.0577
20	20	1.00	20	0.0393	0.0990	0.0746	0.1395	0.0569
20	15	0.75	20	0.0377	0.0943	0.0742	0.1381	0.0560
40	15	0.75	10	0.0386	0.0875	0.0722	0.1301	0.0554
40	15	1.00	10	0.0352	0.0880	0.0742	0.1298	0.0547
40	20	0.75	10	0.0373	0.0884	0.0716	0.1338	0.0544
20	15	0.75	10	0.0379	0.0860	0.0682	0.1311	0.0531
40	20	1.00	10	0.0366	0.0907	0.0696	0.1334	0.0531
20	15	1.00	10	0.0379	0.0844	0.0662	0.1311	0.0521
20	20	0.75	10	0.0366	0.0873	0.0676	0.1371	0.0521
20	20	1.00	10	0.0348	0.0884	0.0662	0.1368	0.0505

The best configuration is `window=40`, `negative=20`, `ns_exponent=0.75`, `epochs=20`, achieving an average HR@10 of 0.0660. Configurations with `epochs=20` consistently outperform those with `epochs=10`, and `window=40` generally outperforms `window=20`, confirming the findings from the single-parameter experiments.

3.5 Final Model

Based on the grid search results, the final model is trained with the following configuration:

Table 12: Final model hyperparameter configuration.

Parameter	Value
Architecture (sg)	Skip-gram (1)
Vector size	100
Window size	40
Min count	5
Negative samples	20
ns_exponent	0.75
Epochs	20
Workers	4

Training completed in 518.6 seconds on 37,704 training playlists, yielding a vocabulary of 54,651 unique tracks. Figure 6 shows the loss progression across epochs.

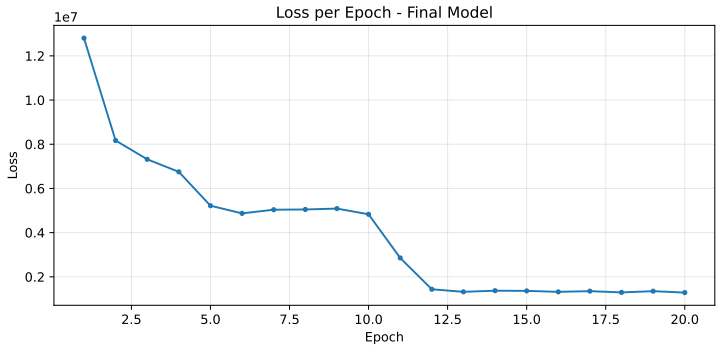

Figure 6: Final model loss per epoch.

The loss decreases rapidly in the first six epochs, enters a plateau between epochs 7–10, then drops sharply around epoch 11–12 before converging stably from epoch 13 onward. This two-phase convergence is a known behavior of Gensim's learning rate schedule, which decreases the learning rate linearly over training.

3.6 Evaluation

3.6.1 Design Decisions

Two deliberate design choices were made during evaluation. First, context songs are not excluded from the recommendation list. Although filtering out context songs is semantically motivated because a user should not be recommended songs they are currently listening to but we observed that 1.36% of playlists contain the true last song as a duplicate within the context. Filtering context songs in these cases removes the correct answer from the candidate list, artificially lowering HR@K. We therefore adopt the unfiltered evaluation to avoid penalizing the model for data artifacts.

Second, duplicate tracks within playlists are not removed. Deduplication would alter the original playlist structure and could distort the co-occurrence statistics that Word2Vec relies on. Since the duplicate rate is low (1.36%), the impact on embedding quality is negligible.

3.6.2 Metrics

We evaluate next-song prediction using two metrics computed at cutoff $K \in \{5, 10, 20, 50\}$:

- **Hit Rate at K (HR@K):** The proportion of test playlists where the true last song appears in the top-K recommendations. It is a binary metric meaning the model either hits or misses for each test case.

- **Normalized Discounted Cumulative Gain at K (NDCG@K):** A rank-aware extension of HR@K. If the true song is found at rank j in the top-K list, the score is $1/\log_2(j+1)$; otherwise 0. Higher-ranked correct predictions receive higher scores.

3.6.3 Results

Table 13 reports HR@K and NDCG@K for both the baseline and the final model under both evaluation approaches. Out of 9,426 test playlists, 4,970 are skipped under Context Averaging and 6,436 under Single Query, as the true last song or the query song is not present in the vocabulary. The higher skip rate for Single Query is because it requires both the true item and the second-to-last song to be in vocabulary, whereas Context Averaging only requires the true item.

Table 13: Evaluation results for baseline and final model.

		HR@K				**NDCG@K**			
Model	**Approach**	**@5**	**@10**	**@20**	**@50**	**@5**	**@10**	**@20**	**@50**
Baseline	Context Avg.	0.0166	0.0249	0.0350	0.0503	0.0138	0.0164	0.0190	0.0220
Baseline	Single Query	0.0214	0.0331	0.0502	0.0706	0.0143	0.0181	0.0224	0.0265
Final	Context Avg.	0.0281	0.0491	0.0747	0.1192	0.0178	0.0246	0.0311	0.0399
Final	Single Query	0.0559	0.0786	0.1107	0.1515	0.0373	0.0446	0.0526	0.0606

The final model substantially outperforms the baseline across all K values and both approaches. Under Single Query at $K = 10$, HR@10 improves from 0.0331 to 0.0786 showing a 2.4× improvement. The two hyperparameter changes that contributed most to this gain are window size ($5 \rightarrow 40$) and number of epochs ($5 \rightarrow 20$): larger windows capture broader playlist context, providing richer co-occurrence signal, while more epochs allow the model to converge to a better embedding space.

Single Query consistently outperforms Context Averaging across all K values and both models. Three factors explain this:

Information loss from averaging. Context Averaging computes the mean of all $N - 1$ context song embeddings. When the playlist contains many songs from different genres or artists, the resulting vector becomes diffuse and loses the specificity needed to identify a single target song.

Task difficulty. Context Averaging must implicitly encode the entire listening history into one vector, which is a harder compression problem than single-song retrieval.

User intent modeling. The second-to-last song in a playlist is the most immediate context for the next song. It reflects the user's current listening mood more directly than the entire playlist history.

The gap between the two approaches widens in the final model, suggesting that larger window sizes and more training epochs amplify the specificity advantage of the Single Query approach.

4 Analysis

Recommendation Interface

We implement a `recommend_songs` function that takes a list of seed songs and a trained model, and returns top-K recommendations. The function computes the average embedding of all in-vocabulary seed songs and retrieves the most similar songs, excluding the seeds from the output. OOV seeds are skipped with a warning; if all seeds are OOV, an empty list is returned.

4.1 Qualitative Analysis

Most Similar Songs

Table 14 shows the top-10 most similar songs for three selected seed tracks, retrieved using `model.wv.most_similar()`.

Table 14: Top-10 most similar songs for three seed tracks.

Rank	Song	Score
Seed: michael jackson__thriller		
1	michael jackson__bad	0.8551
2	michael jackson__beat it	0.8320
3	michael jackson__billie jean	0.8137
4	michael jackson__smooth criminal	0.7944
5	michael jackson__the way you make me feel	0.7441
6	michael jackson__black or white	0.7438
7	michael jackson__someone in the dark	0.7187
8	michael jackson__man in the mirror	0.7183
9	michael jackson__just good friends	0.7022
10	michael jackson__invincible	0.6980
Seed: eminem__lose yourself		
1	50 cent__wanksta	0.6872
2	d12__how come	0.6680
3	eminem__white america	0.6632
4	50 cent__21 questions	0.6541
5	eminem__cleanin' out my closet	0.6532
6	junior stress__znam ten stan	0.6505
7	eminem__mockingbird	0.6491
8	eminem__superman	0.6485
9	kanye west__hell of a life	0.6477
10	eminem__stan	0.6454
Seed: led zeppelin__stairway to heaven		
1	led zeppelin__whole lotta love	0.7674
2	led zeppelin__black dog	0.7663
3	led zeppelin__immigrant song	0.7502
4	led zeppelin__kashmir	0.7255
5	led zeppelin__since i've been loving you	0.6953
6	led zeppelin__heartbreaker	0.6851
7	queen__bohemian rhapsody	0.6849
8	pink floyd__comfortably numb	0.6755
9	pink floyd__wish you were here	0.6610
10	deep purple__soldier of fortune	0.6482

The results reveal two distinct patterns. For Michael Jackson, all recommendations come from the same artist, illustrating the artist over-representation problem discussed in the failure analysis. For Eminem, cross-artist recommendations from the same genre (50 Cent, D12, Kanye West) appear alongside same-artist tracks, with one cross-language recommendation (`junior stress__znam ten stan`). For Led Zeppelin, the model successfully identifies cross-artist classic rock connections, recommending Queen and Pink Floyd alongside Led Zeppelin tracks.

Song Algebra

Similar to the word2vec "king − man + woman = queen" example, we demonstrate song algebra using vector arithmetic on the learned embeddings. Table 15 shows four examples.

Table 15: Song algebra examples using vector arithmetic.

Operation	Top result
`radiohead__creep` − `radiohead__karma police` + `the beatles__let it be`	the beatles__yesterday (0.7576)
`queen__bohemian rhapsody` − `queen__we will rock you` + `led zeppelin__stairway to heaven`	led zeppelin__black dog (0.6977)
`michael jackson__thriller` − `michael jackson__bad` + `eminem__lose yourself`	eminem__my name is (0.5982)
`miles davis__so what` − `miles davis__all blues` + `johnny cash__hurt`	nick cave__suzanne (0.5956)

The first two examples demonstrate successful artist transfer: removing the Radiohead context and adding Beatles shifts the result entirely to Beatles songs, and removing a Queen song and adding Led Zeppelin yields Led Zeppelin tracks. The third example successfully transfers from pop to hip-hop. The fourth example is less precise. `johnny cash__desperado` (rank 3) is actually an Eagles song covered by Cash, illustrating a limitation of the model in distinguishing original recordings from covers.

t-SNE Visualization

Figure 7 and Figure 8 show t-SNE projections of song embeddings for a selected set of artists, colored by artist and genre respectively.

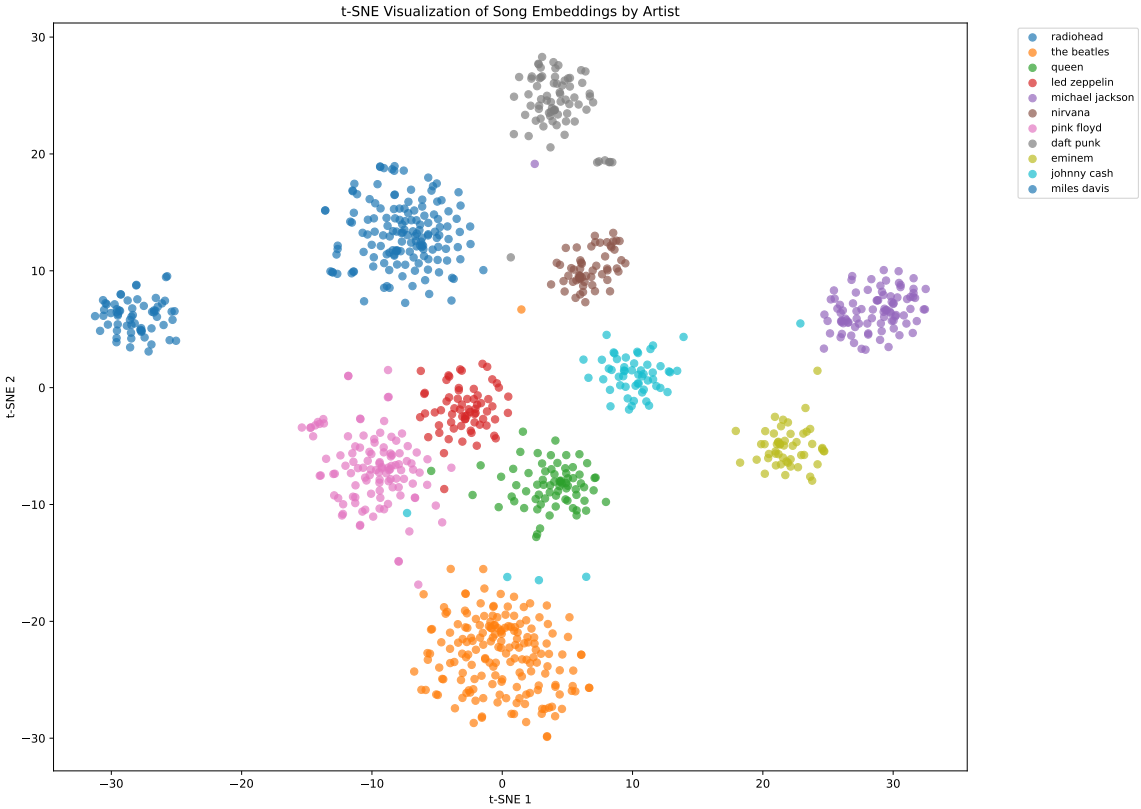

Figure 7: t-SNE visualization of song embeddings colored by artist.

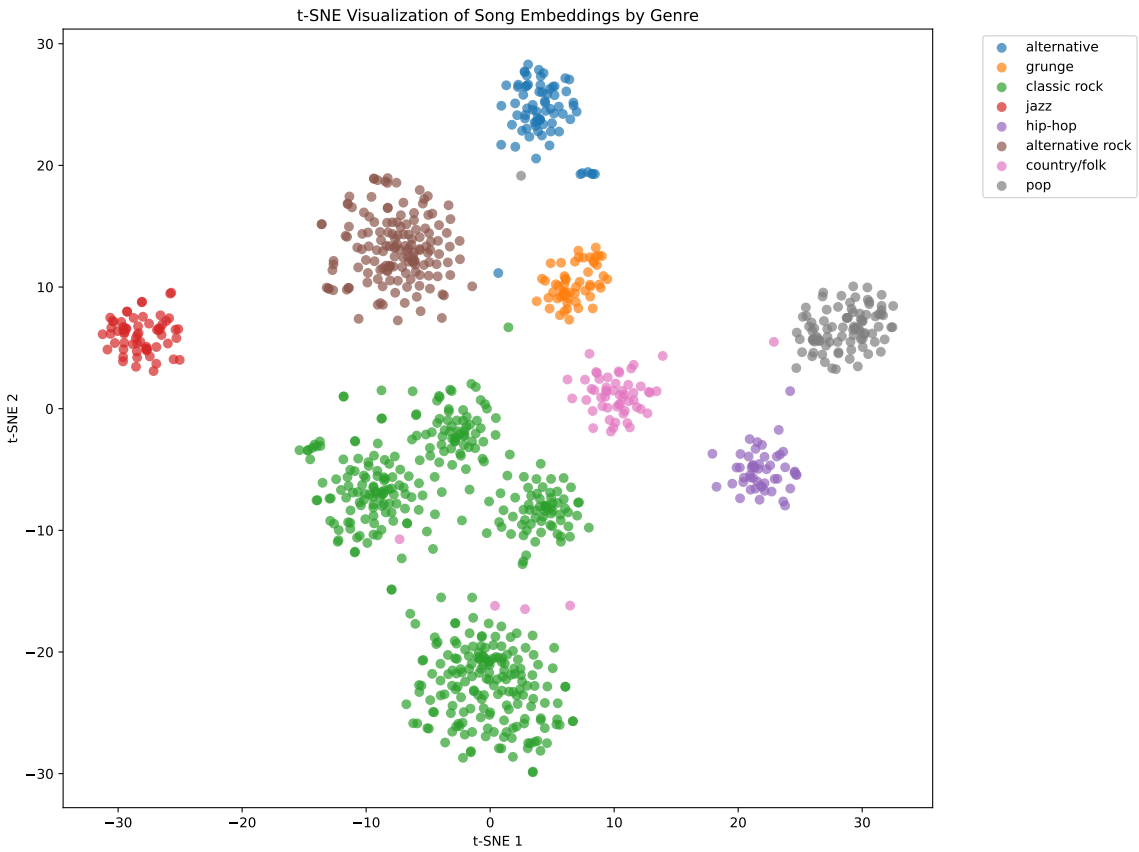

Figure 8: t-SNE visualization of song embeddings colored by genre.

The artist-colored plot shows that songs from the same artist tend to cluster together, confirming that the model captures strong artist-level similarity. The genre-colored plot reveals broader structure: classic rock artists (Queen, Led Zeppelin, Pink Floyd) cluster in overlapping regions, while jazz (Miles Davis) and hip-hop (Eminem) occupy distinct areas of the embedding space. This demonstrates that the model learns not only artist identity but also higher-level genre relationships from playlist co-occurrence patterns.

4.2 Cold-Start Analysis

Vocabulary Exclusion

With `min_count=5`, 86.15% of unique training tracks (339,859 out of 394,510) are excluded from the vocabulary. These tracks appear fewer than 5 times in the training set and therefore lack sufficient co-occurrence data for meaningful embedding learning.

OOV Rate on Test Set

Table 16 shows the OOV breakdown for the test set. Of the 162,224 unique test tracks, 73.48% are out-of-vocabulary. This OOV rate has two distinct sources: tracks that never appeared in the training set (cold-start), and tracks that appeared in training but were filtered out by `min_count`.

Table 16: OOV breakdown for the test set.

Category	**Count (%)**
Total unique test tracks	162,224 (100.00%)
In-vocabulary	43,015 (26.52%)
OOV total	119,209 (73.48%)
↪ Unseen in train	48,674 (30.00%)
↪ Seen but below min_count	70,535 (43.48%)

The majority of OOV tracks (43.48%) were seen during training but discarded by the `min_count` threshold, rather than being truly unseen. This suggests that lowering `min_count` could recover a significant portion of OOV cases, at the cost of embedding quality as discussed in Vocabulary Coverage part in Discussion.

Artist-Level Fallback

To partially address the cold-start problem, we implement an artist-level fallback strategy: when a seed song is OOV, the average embedding of all known songs by the same artist is used as a proxy. This is feasible because the artist index contains 10,858 artists with at least one track in the vocabulary.

Table 17 demonstrates the effect of the fallback on a concrete example where `daft punk__get lucky` is OOV.

Table 17: Recommendation comparison with and without artist-level fallback for an OOV seed.

Rank	Without fallback	Rank	With fallback
1	queen__we are the champions	1	david guetta__distortion
2	queen__we will rock you	2	queen__we are the champions
3	junior stress__znam ten stan	3	daft punk__da funk
4	queen__innuendo	4	queen__we will rock you
5	50 cent__wanksta	5	50 cent__wanksta

Without fallback, the OOV seed is silently ignored and recommendations are driven entirely by the remaining seeds. With fallback, electronic music recommendations (David Guetta, Daft Punk) appear in the list, partially recovering the lost genre context.

Frequent vs. Rare Songs

Table 18 compares HR@10 for test playlists ending with a frequent song (> 100 occurrences in training) versus a rare song (< 10 occurrences) using the final model.

Table 18: Final model HR@10 for frequent vs. rare songs.

Group	Approach	N	HR@10
Frequent (> 100)	Context Avg.	744	0.1304
Frequent (> 100)	Single Query	496	0.1190
Rare (< 10)	Context Avg.	660	0.0485
Rare (< 10)	Single Query	383	0.0601

Frequent songs achieve roughly 2–3× higher HR@10 than rare songs across both approaches. This gap reflects the embedding quality difference: frequent tracks appear in many playlists and accumulate rich co-occurrence signal, while rare tracks have sparse training data and correspondingly weaker embeddings.

4.3 Failure Analysis

Table 19 presents six failure cases identified in our system. For two of them (Failures 1 and 2), we propose and implement a fix.

Table 19: Failure analysis: bad cases observed in the Song2Vec recommendation system.

#	Input/Query	Model Output	Expected	Error Type	Root Cause
1	`queen__bohemian rhapsody`	10/10 Queen songs	Cross-artist classic rock recommendations	Artist over-representation	Same-artist songs co-occur heavily in playlists, clustering their embeddings tightly together
2	`daft punk__get lucky`	OOV — skipped entirely	Daft Punk / electronic songs	Cold-start failure	Track appears after dataset collection period; absent from vocabulary
3	Mixed seeds: Queen + Eminem + Daft Punk	Classic rock dominant recommendations	Balanced cross-genre recommendations	Genre drift	Dominant genre pulls the average seed vector toward its own cluster
4	`zz top__she loves my automobile` (31,036 occurrences)	Obscure, unrelated songs (freq ≤ 8)	Popular classic rock songs	Embedding dilution	Extreme frequency causes the embedding to average over too many diverse contexts, losing specificity
5	Mixed seeds from 4 different genres	Mean similarity score: 0.64	Mean similarity score > 0.75	Low confidence	Average vector of unrelated genres lands in a low-density region of the embedding space
6	`eminem__lose yourself`	`junior stress__znam ten stan` (Polish) at rank 6	English hip-hop songs	Cross-language recommendation	Model learns co-occurrence without language awareness; Polish hip-hop co-occurs with Eminem in multilingual playlists

Fix 1: Artist Diversity Filter (Failure 1)

To address artist over-representation, we apply a post-processing filter that limits recommendations to at most m songs per artist. After retrieving the top-K candidates by similarity, we iterate through the list and skip any song whose artist has already appeared m times.

Before fix: Querying `queen__bohemian rhapsody` returns 10/10 Queen songs.

After fix ($m = 2$): Returns 2 Queen songs followed by Led Zeppelin, Accept, Eagles, Deep Purple, Guns N' Roses, Dire Straits, and Lynyrd Skynyrd — all within the classic rock genre, preserving semantic coherence while improving diversity.

Fix 2: Artist-Level Fallback (Failure 2)

To handle OOV seeds, we implement an artist-level fallback: when a seed song is not in the vocabulary, we compute the average embedding of all known songs by the same artist and use it as a proxy embedding.

Before fix: `daft punk__get lucky` is silently skipped; recommendations are shaped only by the remaining seeds, with no electronic music influence.

After fix: The artist embedding of Daft Punk is used as a proxy. `david guetta__distortion` and `daft punk__da funk` appear in the recommendation list, partially recovering the electronic music context.

5 Discussion

5.1 Hyperparameter Analysis

In our experiments, `ns_exponent = 0.75` (NLP default) outperformed `ns_exponent = -0.5` (recommendation setting) on both evaluation approaches. The exact HR@10 values are reported in Table 20.

Table 20: HR@10 comparison for ns_exponent = 0.75 vs. -0.5.

Approach	**ns_exponent = 0.75**	**ns_exponent = -0.5**
Context Averaging	0.0238	0.0200
Single Query	0.0355	0.0227

Caselles-Dupré et al. [1] recommend negative `ns_exponent` values for recommendation tasks, arguing that sampling unpopular items as negatives improves performance on long-tail items. However, this finding was obtained on listening session data where a small number of songs dominate with millions of occurrences (log count up to 7). In our playlist dataset, the popularity distribution is considerably flatter: only one track exceeds 1,000 occurrences, and only 723 tracks exceed 100 occurrences, with a maximum of 31,036. Under this distribution, sampling unpopular items as negatives does not provide a meaningful learning signal, since the gap between popular and unpopular items is much smaller. In contrast, `ns_exponent = 0.75` samples popular items more frequently as negatives, which better discriminates between similar embeddings and yields higher HR@10 on our data.

5.2 Playlist Length Effect

We split the test playlists into three groups based on their length: short ($<$ 10 songs), medium (10–30 songs), and long ($>$ 30 songs). Table 21 reports HR@10 for each group under both evaluation approaches.

Table 21: HR@10 by playlist length group.

Group	**Playlists**	**Avg. length**	**Context HR@10**	**Single HR@10**
Short ($<$ 10)	3,169	5.0	0.0762	0.0870
Medium (10–30)	3,337	17.2	0.0475	0.0638
Long ($>$ 30)	2,920	83.4	0.0249	0.0870

For Context Averaging, short playlists achieve the highest HR@10 (0.0762), followed by medium (0.0475) and long (0.0249). This monotonic decrease is reasonable because as the number of context songs grows, averaging their embeddings produces an increasingly diffuse query vector that loses specificity. It makes it harder to identify the correct next song.

For Single Query, short and long playlists achieve identical HR@10 (0.0870), while medium playlists perform lower (0.0638). This is consistent with the nature of the Single Query approach since only the second-to-last song is used as the query regardless of playlist length. Performance is largely independent of how long the playlist is. The lower score for medium playlists is likely attributable to statistical variation rather than a systematic effect.

Overall, short playlists show the best or equal performance across both approaches. This suggests that next-song prediction is easier when fewer context songs are available, as the prediction task remains more constrained and specific.

5.3 Vocabulary Coverage

With `min_count=5`, the vocabulary contains 54,651 unique tracks. The test set contains 162,224 unique tracks, of which 73.48% are out-of-vocabulary (OOV). Table 22 shows how `min_count` affects vocabulary size, OOV rate, and HR@10.

Table 22: Effect of min_count on vocabulary size, test OOV rate, and HR@10. Total unique test tracks: 162,224.

min_count	Vocab size	Test OOV	Test OOV (%)	Context HR@10	Single HR@10
3	106,476	96,513	59.49%	0.0178	0.0229
5	54,651	119,209	73.48%	0.0251	0.0314
10	21,430	141,950	87.50%	0.0441	0.0656

Higher `min_count` reduces vocabulary size and increases OOV rate, yet HR@10 improves. This is because tracks with very few occurrences do not have enough co-occurrence data for meaningful embeddings. Thus, including them adds noise. With `min_count=10`, only the most frequent tracks are retained, producing cleaner embeddings and higher HR@10.

The tradeoff is coverage: at `min_count=10`, 87.50% of test tracks are OOV and cannot be evaluated. We choose `min_count=5` as a balance between embedding quality and test coverage.

5.4 Popular vs. Rare Songs

Table 23 compares HR@10 for test playlists ending with a frequent song (> 100 occurrences in training) versus a rare song (< 10 occurrences), for both the baseline and the final model.

Table 23: HR@10 for frequent vs. rare songs — baseline vs. final model.

Group	Approach	N	Baseline HR@10	Final HR@10	Improvement
Frequent (> 100)	Context Avg.	744	0.0833	0.1304	+0.0471
Frequent (> 100)	Single Query	496	0.0685	0.1190	+0.0505
Rare (< 10)	Context Avg.	660	0.0000	0.0485	+0.0485
Rare (< 10)	Single Query	383	0.0026	0.0601	+0.0575

The gap between frequent and rare songs is large in both models. In the baseline, the model completely fails on rare songs under Context Averaging (HR@10 = 0.0000) and nearly fails under Single Query (HR@10 = 0.0026). This is expected: rare songs appear fewer than 10 times in training, leaving Word2Vec with insufficient co-occurrence data to learn reliable embeddings.

The final model improves performance on both groups. Notably, rare song performance improves substantially from 0.0000 to 0.0485 (Context Averaging) and from 0.0026 to 0.0601 (Single Query). This improvement is largely driven by the larger window size (`window=40`), which captures broader playlist context and provides more training signal for rare songs. Despite this, the gap between frequent and rare songs remains significant, confirming that the cold-start problem for rare items is not fully resolved by hyperparameter tuning alone.

5.5 Error Patterns

The most common error type observed in our failure analysis is **artist over-representation**: when a seed song belongs to a prolific artist with many tracks in the vocabulary, the model returns recommendations dominated by other songs from the same artist. For example, querying `queen__bohemian rhapsody` returns 10 out of 10 Queen songs, and similar behavior is observed for Radiohead, Metallica, and Michael Jackson.

This occurs because Word2Vec learns embeddings based on co-occurrence in playlists. Songs by the same artist naturally co-occur more frequently than songs by different artists, causing their embeddings to cluster tightly together. When the nearest neighbors are retrieved, songs from the same artist dominate simply because they are geometrically closer in the embedding space.

Proposed solution: Artist diversity filter. After retrieving the top-K candidates, apply a post-processing step that limits the number of recommendations from any single artist to a maximum of m songs (e.g., $m = 2$). The remaining slots are filled by the next highest-scoring candidates from different artists.

We implemented this filter and demonstrated its effect on `queen__bohemian rhapsody`:

- **Before fix:** 10/10 recommendations from Queen.
- **After fix ($m = 2$):** 2 Queen songs + 8 songs from Led Zeppelin, Accept, Eagles, Deep Purple, Guns N' Roses, Dire Straits, and Lynyrd Skynyrd which are all within the classic rock genre, preserving semantic coherence while improving diversity.

This solution is effective because it directly addresses the geometric clustering problem without modifying the embedding model. It is also computationally cheap since it requires only a single pass over the ranked list and can be tuned via the m parameter depending on the desired diversity level.

References

[1] Hugo Caselles-Dupré, Florian Lesaint, and Jordi Royo-Letelier. Word2vec applied to recommendation: Hyperparameters matter. In *Proceedings of the 12th ACM Conference on Recommender Systems*, RecSys '18, pages 352–356, New York, NY, USA, 2018. ACM.

[2] Roberto Turrin, Massimo Quadrana, Andrea Condorelli, Roberto Pagano, and Paolo Cremonesi. 30music listening and playlists dataset. In *Proceedings of the ACM RecSys 2015 Workshop on Recommender Systems and the Social Web*, 2015.